



Backend Engineer

Background

You have recently joined LOOP DFS as a Backend Engineer within the Digital Business team.

The team has received a request from multiple channels including:

- Mobile Applications
- Web Applications
- Partner APIs
- Internal Operations Portals

These systems require access to country, currency, language and geographical reference data.

Currently every channel consumes a third-party SOAP service directly.

This has resulted in several operational challenges:

- Inconsistent responses across channels
- Poor performance due to repeated SOAP calls
- Limited filtering capabilities
- No pagination support
- Lack of auditability

- No centralized caching strategy
- SOAP credentials exposed across multiple applications

The architecture team has approved the development of a new service called:

Reference Data Aggregation Service (RDAS)

RDAS will become the single source of truth for all country reference information.

The service must expose REST/JSON APIs while consuming the existing SOAP service internally.

External Integration

The Reference Data Aggregation Service (RDAS) must integrate with the CountryInfo SOAP Service.

The service exposes a number of SOAP operations that may be used to satisfy the business requirements.

WSDL Endpoint

<http://webservices.oorsprong.org/websamples.countryinfo/CountryInfoService.wso?WSDL>

Available SOAP Operations

Operation	Description
CapitalCity	Returns a country's capital city
CountriesUsingCurrency	Returns countries using a specified currency
CountryCurrency	Returns country currency information

Operation	Description
CountryFlag	Returns country flag URL
CountryIntPhoneCode	Returns international dialing code
CountryName	Returns country name
CurrencyName	Returns currency name
LanguageName	Returns language name
ListOfContinentsByName	Returns available continents
ListOfCountryNamesGroupedByContinent	Returns countries grouped by continent
ListOfCurrenciesByName	Returns available currencies
ListOfLanguagesByName	Returns available languages

Business Requirement

A consuming application wants to search for countries using a single API.

The application should be able to:

- Search by country name
- Filter by continent
- Filter by currency
- Filter by language
- Retrieve country details
- View countries sharing the same currency
- Support pagination
- Support sorting

The SOAP service does not provide a single operation capable of meeting these requirements.

Your task is to design and implement a solution.

Part 1 – Solution Design

Document your proposed architecture.

Part 2 – API Design

Design the APIs required to satisfy the business requirement.

Part 3 – Data Processing Challenge

The SOAP provider has introduced a usage limit of:

100 requests per minute

Explain:

- How your design reduces SOAP traffic
- What data should be cached
- Cache expiration strategy
- Cache refresh strategy

Provide justification for your design decisions.

Part 4 – Resilience Challenge

The SOAP service becomes unavailable for 6 hours.

Describe:

- What happens when a request arrives
- How users should experience the failure
- What fallback mechanisms should exist
- What monitoring and alerting should be triggered

Part 5 – Implementation

Implement the APIs designed in Part 2.

Requirements:

- Pagination
- Sorting
- Filtering
- Caching
- Global Error Handling
- Input Validation

Technology stack:

- Spring Boot 3+
- Java 17+
- Maven

Part 6 – Engineering Discussion

Provide written responses to the following:

Question 1

How would your solution change if the SOAP provider reduced the limit to 10 requests per minute?

Question 2

How would you scale the solution to support 20 million requests per day?

Question 3

What additional improvements would you make if given another week to enhance the platform?

Deliverables

Submit:

1. Source Code on github and share the link
2. Architecture Diagram
3. API Documentation
4. README
5. Create deployment scripts that will be used to deploy the application above on kubernetes.
6. Create a guide on how to deploy the application on kubernetes.
7. Provide a guide on how to troubleshoot the application on kubernetes